Data Collection and Imitation Learning

Course home · 简体中文 · PDF

Item	Details
Applicability	Demonstrations, datasets, BC, DAgger, ACT, generative action chunks and RTC concepts; no single tested software environment is prescribed
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Document integration and rendering checks only; no tutorial runtime, training or hardware validation. See the materials manifest for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Robot Demonstrations and Data Collection
  - 1.1 From trajectories to a policy-learning problem
  - 1.2 Demonstration sources and trade-offs
  - 1.3 Minimum information in a useful episode
- 2. Dataset Representation, Synchronization, and Quality
  - 2.1 Clocks and causal alignment
  - 2.2 Frames, units, and normalization
  - 2.3 Storage schema and dataset curation
- 3. Imitation Learning: From Behavior Cloning to DAgger
  - 3.1 What does imitation learning actually learn?
  - 3.2 Behavior cloning: choosing a supervised objective
  - 3.3 Why squared error can favor the wrong action
  - 3.4 Covariate shift and compounding error
  - 3.5 DAgger: the complete data-aggregation loop
  - 3.6 The no-regret perspective and the limits of its guarantees
  - 3.7 Expert interfaces for DAgger on real robots
  - 3.8 How BC, DAgger, and generative IL fit together
- 4. Action Chunks, ACT, Diffusion, and Flow Matching
  - 4.1 Prediction horizon versus execution horizon
  - 4.2 ACT: Action Chunks, a Conditional VAE, and Transformers
  - 4.3 Diffusion Policy: iterative denoising
  - 4.4 Flow Matching: a velocity field in generation space
  - 4.5 Choosing a policy family

- 5. Real-Time Action Execution and RTC
  - 5.1 Two clocks: physical control and model inference
  - 5.2 Synchronous execution, naive asynchrony, Temporal Ensembling, and RTC
  - 5.3 The RTC timeline: committed prefix and unconstrained suffix
  - 5.4 Inference-time RTC: generate a new chunk through inpainting
  - 5.5 Training-time RTC: learn to condition directly on the action prefix
  - 5.6 Runtime components needed by an RTC executor
  - 5.7 What RTC does not solve: contact, perception, and coverage
  - 5.8 Concept map
- 6. Augmentation, Retargeting, and Recovery
  - 6.1 Physically consistent augmentation
  - 6.2 The missing information in human-video retargeting
  - 6.3 Learning from failure without copying failure
- 7. Deployment Interfaces and Course Boundaries
  - 7.1 Modular contracts
  - 7.2 Diagnosing common failure modes
  - 7.3 Relationship to other course modules
- References and Further Reading

> **Scope.** These notes explain demonstrations, dataset construction, imitation objectives, generative action chunks, real-time execution, data augmentation, and correction. There is **no experimental chapter, protocol, or ablation plan**. Detailed calibration, reinforcement learning, and flow-matching theory belong in companion courses.
>
> **Learning path.** Collect → curate → BC objectives and covariate shift → DAgger aggregation → ACT and generative chunks → RTC timeline and prefix conditioning → correction/recovery → deployment interfaces.

1. Robot Demonstrations and Data Collection

1.1 From trajectories to a policy-learning problem

Imitation learning is not frame-by-frame video copying. It learns a conditional policy from the states visited and actions selected by a demonstrator:

$$\pi_\theta(a_t \mid o_{\le t}, q_{\le t}, g),$$

where o_t is exteroceptive observation, q_t is proprioception, g is the task condition, and a_t denotes actions under a specified control interface. Physical dynamics can be written as $s_{t+1} \sim P(\cdot \mid s_t, u_t)$ and $o_t \sim O(\cdot \mid s_t)$. **Latent physical state s_t, sensor observation o_t, predicted action a_t, issued command u_t, and measured feedback are not interchangeable.** Safety filtering, saturation, and communication delays may make $u_t \neq a_t$.

An episode can be represented as

$$\tau_i = \{(t_k, o_k, q_k, a_k, u_k, \hat{q}_{k+1}, g, e_k)\}_{k=0}^{T_i-1},$$

where e_k marks contact, slip, takeover, failure, or recovery. Episode-level success is a separate label; it does not replace step-level action supervision.

1.2 Demonstration sources and trade-offs

Source	Typical action label	Benefit	Limitation
Kinesthetic teaching	Joint position/velocity target	Directly tied to robot kinematics	High demonstrator burden; contact forces may vary
Leader–follower teleoperation (GELLO, ALOHA)	Follower commands	Captures real contact and long sequences	Mapping, latency, workspace limitations
Portable interfaces (UMI)	Tool/end-effector motion and vision	Collection across varied environments	Embodiment-specific retargeting required
Simulator or scripted expert	Simulated control signals	Scalable and automatically labeled	Reality gap in vision and contact
Human-video reconstruction	Inferred hand/object trajectories	No target robot needed at collection	Occluded actions and forces are ambiguous

See GELLO, ALOHA / ACT, and UMI. Their action semantics are different even if the resulting arrays have identical dimensions.

1.3 Minimum information in a useful episode

Maintain task description, image sequences with camera-specific timestamps, joint positions and velocities, end-effector poses, control mode, predicted actions, sent commands, observed feedback, hand/gripper state, calibration version, hardware configuration, and event annotations. When force control is involved, capture torque/force or tactile modalities with units, scale, and sampling frequency.

Explicitly type each action: $a_t = q^{\text{target}}$, Δx^{EE}, $\dot{q}^{\text{cmd}}$, or τ^{cmd}. A position increment interpreted as an absolute target during execution is a representation failure, not a policy-learning failure.

2. Dataset Representation, Synchronization, and Quality

2.1 Clocks and causal alignment

Camera, robot-state, and tactile streams often run at different rates. Distinguish exposure time, message reception, decision time, and physical command execution. For a policy action at time t, a causally available observation can be selected as

$$\tilde{o}_t = o_{k^*(t)}, \qquad k^*(t) = \max\{k : t_k^{\text{available}} \le t\}.$$

Using a future video frame to interpolate an earlier observation silently leaks information into training. Record delay explicitly instead of hiding it inside resampling.

2.2 Frames, units, and normalization

Use a consistent convention $T_B^A \in SE(3)$ mapping B-frame coordinates to frame A. Include reference frames and units for every pose, velocity, and increment, and specify left- versus right-multiplicative SE(3) updates. Distinguish camera, base, end-effector, and task frames; state whether extrinsics remain static or vary with robot configuration.

A common training-only normalization is

$$\bar{a}_j = \frac{a_j - \mu_j}{\sigma_j + \epsilon}, \qquad a_j = \bar{a}_j(\sigma_j + \epsilon) + \mu_j.$$

Persist the training split's statistics and their schema version. Do not normalize absolute positions, angles, rotation representations, and binary gripper commands indiscriminately with identical rules.

2.3 Storage schema and dataset curation

Organize data using episode metadata, video indices, timestamped vector fields, typed actions, and format versions. LeRobot Dataset v3 provides a concrete organizational reference, but version-lock schemas for long-lived datasets.

Quality gates should catch nonmonotonic timestamps; dropped frames; unavailable calibration; large action/feedback mismatch; illegal joint values; corrupted videos; missing or ambiguous contact events; and duplicate idle segments. Retain informative failed or recovered episodes while rejecting invalid demonstrations arising from communication faults.

Split by **scene, object instance, operator, initial condition, or collection session**, not by random individual frames. Adjacent train and test images from the same episode will overstate generalization.

3. Imitation Learning: From Behavior Cloning to DAgger

3.1 What does imitation learning actually learn?

Imitation learning (IL) seeks a deployable closed-loop policy from expert behavior, without first specifying a task reward or solving a complete dynamics model. Let s_t be the hidden physical state, $h_t = (o_{\le t}, q_{\le t}, g)$ the available observation history, and u_t the command executed by the robot:

$$s_{t+1} \sim P(\cdot \mid s_t, u_t), \qquad h_t \sim \mathcal{O}(\cdot \mid s_{0:t}, u_{0:t-1}), \qquad a_t \sim \pi_\theta(\cdot \mid h_t).$$

The policy may predict an individual action or a future action sequence. With occlusion, hidden contacts, and sensing delay, the current image alone need not be Markovian. Adding an observation history, proprioception, or estimated state can mitigate partial observability, but does not guarantee recovery of the full physical state.

For an episode of T steps, define time-dependent and average state occupancy distributions by

$$d_t^\pi(s) = \Pr(s_t = s \mid \pi), \qquad d^\pi(s) = \frac{1}{T}\sum_{t=0}^{T-1} d_t^\pi(s).$$

The central challenge is that demonstrations sample the expert-induced occupancy d^{π_E}, whereas deployment visits d^{π_θ}. BC addresses how to fit expert actions; DAgger additionally addresses which actions should be taken in states caused by the learner itself.

Useful categories are **offline imitation** (a fixed dataset, typically BC), **interactive imitation** (new expert labels on learner-visited states, typically DAgger), and **inverse reinforcement learning** (inferring a reward from behavior, a separate direction). These notes focus on the first two. ACT, Diffusion Policy, and Flow Matching can all serve as action-modeling families trained with a behavior-cloning signal.

3.2 Behavior cloning: choosing a supervised objective

Given $\mathcal{D}_E = \{(h_i, a_i^E)\}_{i=1}^N$, deterministic BC can minimize weighted squared error:

$$\pi_\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \|\pi_\theta(h_i) - a_i^E\|_W^2, \qquad \|x\|_W^2 = x^\top W x,\ W \succeq 0.$$

This works best when the expert action conditioned on h is approximately unimodal. The matrix W balances action coordinates, but **cannot substitute for correct physical units or normalization**. Squared error corresponds to the negative log-likelihood of a Gaussian with fixed covariance. For example, if

$$p_\theta(a \mid h) = \mathcal{N}\big(a; \mu_\theta(h), \Sigma\big),$$

then, up to terms independent of the network,

$$\mathcal{L}_{\mathrm{NLL}} = \mathbb{E}_{\mathcal{D}_E}\left[\tfrac{1}{2}(a^E - \mu_\theta(h))^\top \Sigma^{-1}(a^E - \mu_\theta(h))\right].$$

When the model predicts the covariance, its $\log\det\Sigma_\theta(h)$ contribution must also be retained, or the model could reduce loss merely by inflating uncertainty. An L_1 loss can be associated with a Laplace noise model; genuinely discrete gripper modes can be modeled with cross-entropy. A continuously controlled gripper position should not be converted to a binary label without justification.

At the population level, probabilistic BC can be formulated as

$$\min_\theta\ \mathbb{E}_{h\sim d^{\pi_E}} D_{\mathrm{KL}}(\pi_E(\cdot \mid h) \| \pi_\theta(\cdot \mid h)).$$

Under adequate expert action samples and sufficient model capacity, empirical negative log-likelihood approximates this objective. Crucially, it optimizes errors under **expert occupancy**, not the occupancy induced by the learned policy.

Three implementation principles matter: compute normalizers only on the training partition; causally align sensor observations with expert actions; and mask padding in variable-length episodes. With $m_{i,k} \in \{0,1\}$ denoting whether an action target exists,

$$\mathcal{L}_{\mathrm{masked}} = \frac{\sum_{i,k} m_{i,k}\, \ell(\hat{a}_{i,k}, a_{i,k}^E)}{\sum_{i,k} m_{i,k} + \varepsilon}.$$

Unmasked zero-padding teaches the model to output artificial zero actions at episode boundaries.

3.3 Why squared error can favor the wrong action

When the same observation admits several valid expert decisions (e.g., moving to either side of an obstacle), the squared-error minimizer is the conditional mean:

$$f^*(h) = \mathbb{E}[a^E \mid h].$$

This mean can lie between valid modes and lead directly into the obstacle. The problem is a **structural mismatch between multimodal expert actions and unimodal regression**, not merely insufficient network capacity.

Possible remedies are to add an instruction or history that disambiguates intention, use a mixture-density or latent-action model, or apply generative BC through diffusion or flow matching. None of these repairs wrong action semantics, sensor/action misalignment, or inadequate coverage of contact states in the underlying dataset.

3.4 Covariate shift and compounding error

Suppose the learner has at most ϵ probability of a consequential action error at each state visited by the expert. If the first error can take the robot far from the expert trajectory, a union-bound argument gives roughly $O(t\epsilon)$ probability that an error has occurred before step t. Summing the possible downstream costs leads to a classical worst-case scaling of

$$\sum_{t=1}^{T} O(t\epsilon) = O(T^2\epsilon).$$

If the policy instead maintains error ϵ **under its own occupancy distribution**, the corresponding cumulative scaling can be $O(T\epsilon)$ under standard finite-horizon assumptions. These are explanatory guarantees with assumptions on the loss, horizon, and expert comparison. They do **not** imply a universal physical success rate of $1 - T^2\epsilon$, and continuous-action MSE cannot be inserted directly as a discrete mistake probability.

For example, a small grasp-position error can change fingertip contact, cause an object to slip, alter subsequent images and joint states, and ultimately move the policy far outside the demonstration distribution. More expert coverage, disturbance recovery demonstrations, and DAgger address different parts of this chain.

3.5 DAgger: the complete data-aggregation loop

DAgger (Dataset Aggregation) is not a new neural architecture. It is a **training protocol that brings states induced by the learner into expert-labeled supervision**. At iteration k, with policy π_k and aggregated dataset $\mathcal{D}_k$:

1. Execute the learner, optionally mixing some expert control, to collect visited histories h_t.
2. Ask the expert what action should be taken at each selected visited history: $a_t^E = \pi_E(h_t)$. This label is **not necessarily the action that was actually executed**.
3. Add (h_t, a_t^E) to the dataset, not merely successful episodes or images of failures.
4. Refit a supervised policy on $\mathcal{D}_{k+1}$ and repeat.

A conventional formulation is

$$
\begin{aligned}
\pi_k^{\text{roll}} &= \beta_k \pi_E + (1-\beta_k)\pi_k, \\
\mathcal{D}_{k+1} &= \mathcal{D}_k \cup \left\{(h_t, \pi_E(h_t)) : h_t \sim d^{\pi_k^{\text{roll}}}\right\}, \\
\pi_{k+1} &\approx \arg\min_{\pi\in\Pi} \sum_{(h,a^E)\in\mathcal{D}_{k+1}} \ell(\pi(h), a^E).
\end{aligned}
$$

The coefficient β_k denotes **probabilistically choosing expert control at a step or episode**, not a numerical interpolation between expert and learner action vectors. Such interpolation can be unsafe across contact-mode changes. In classic DAgger analysis, expert mixing is reduced so that visited states increasingly reflect the learner's own policy; the mixing schedule and expert queries should be recorded explicitly.

The essential distinction from collecting extra demonstrations is **who chooses the visited states** (the learner) and **who provides the desired label** (the expert). Simply including failed learner actions in a BC dataset is not DAgger.

3.6 The no-regret perspective and the limits of its guarantees

Define iteration-k surrogate risk under the policy-induced distribution:

$$
L_k(\pi) = \mathbb{E}_{h\sim d^{\pi_k}}[\ell(\pi(h), \pi_E(h))].
$$

An online learner with regret

$$
R_N = \sum_{k=1}^{N} L_k(\pi_k) - \min_{\pi\in\Pi} \sum_{k=1}^{N} L_k(\pi)
$$

satisfies the average-risk relation

$$
\frac{1}{N}\sum_{k=1}^{N} L_k(\pi_k) \le \epsilon_N + \frac{R_N}{N}, \qquad \epsilon_N = \min_{\pi\in\Pi} \frac{1}{N}\sum_{k=1}^{N} L_k(\pi).
$$

With additional assumptions concerning bounded cost, expert cost-to-go, policy selection, and a sufficiently small expert-mixing contribution, low surrogate error **on learner-visited states** translates into favorable sequential decision performance and avoids the naive BC's worst-case quadratic amplification. Here ϵ_N is the best policy-class approximation error across the visited distributions. If expert actions cannot be queried reliably, supervision is inconsistent, the model class is inadequate, or contact costs are unbounded, the argument may not apply.

Consequently, “no-regret” does not mean DAgger always outperforms BC, nor that every physical manipulation task enjoys one unconditional $O(T\epsilon)$ bound. The result is a conditional online-learning reduction, not a blanket hardware safety certificate.

3.7 Expert interfaces for DAgger on real robots

Expert interface	Available supervision	Important caveat
Offline relabeling	Desired expert action at a learner-visited state	Recorded video may omit hidden contact state
Real-time expert takeover	Safe corrections following intervention	Pre-takeover failures are not expert action labels

Expert interface	Available supervision	Important caveat
Learner execution + simultaneous expert query	Desired action at the same visited history	Annotation timing, latency, and clock alignment

Retain fields such as `executed_action`, `expert_label`, `policy_version`, `intervention_flag`, and label confidence. Some states are not safely labelable: severe occlusion, unrecoverable contacts, or imminent collision may warrant abstention, reset, or recording for a separate safety module instead of routine supervised updating.

DAgger is not reinforcement learning: it needs desired expert actions, not a reward-trained value estimator. Nor is every human-in-the-loop dataset a DAgger dataset: binary success feedback, scalar reward, or preference judgments without expert action labels generally do not meet the standard definition.

3.8 How BC, DAgger, and generative IL fit together

Approach	Visited-state source	Supervision	Primary role
Single-step BC	Fixed expert dataset	Action regression/likelihood	Inexpensive expert imitation
Chunked BC / ACT	Fixed expert dataset	Future action sequences	Temporal consistency and chunked action generation
Diffusion / Flow BC	Fixed expert dataset	Generative distribution objective	Multimodal conditional actions
DAgger with a supervised policy	Learner-visited states over rounds	Expert actions or labeled chunks	Covariate shift and recovery coverage

ACT and DAgger are complementary, not competing alternatives: ACT specifies how to represent and predict action sequences; DAgger specifies which states to label and how to aggregate supervision. Applying DAgger to a chunked policy additionally requires specifying how an expert supplies correctly aligned future action chunks. A single expert action cannot automatically supervise an entire future chunk.

Reference: Ross et al., DAgger (AISTATS 2011).

4. Action Chunks, ACT, Diffusion, and Flow Matching

4.1 Prediction horizon versus execution horizon

An action-chunk policy predicts H future actions from current history:

$$A_t = (a_{t|t}, a_{t+1|t}, \dots, a_{t+H-1|t}), \qquad A_t \sim p_\theta(\cdot \mid h_t).$$

The prediction length H is **not** the number $K \le H$ of actions executed open-loop before replanning. Increasing K can reduce inference demand but usually slows feedback response. ACT, Diffusion Policy, and Flow Matching can share the same chunked output interface while using different distribution models.

4.2 ACT: Action Chunks, a Conditional VAE, and Transformers

4.2.1 The two problems ACT was designed to address

Action Chunking with Transformers (ACT), introduced in the ALOHA work, addresses more than long-range sequence memorization. Its motivations include compounding error in single-step imitation, and variation in human demonstrations such as pauses, different execution speeds, and multiple valid trajectories.

At observation history h_t, let the expert's next H actions form

$$A_t^E = [a_t^E, a_{t+1}^E, \ldots, a_{t+H-1}^E] \in \mathbb{R}^{H \times D_a}.$$

ACT produces $\hat{A}_t = f_\theta(h_t, z)$, where z is a latent variable representing aspects of demonstration style. **The prediction horizon H is not the frequency of sensor feedback**; the execution layer decides how often the policy is queried and how many predictions are used. The original ACT models sequences of **target joint positions** tracked by lower-level controllers. Those tensors cannot simply be reinterpreted as torque, velocity, or delta-position commands without changing the action interface.

4.2.2 What the conditional VAE encoder and decoder do

ACT trains a conditional variational autoencoder (CVAE). Its training encoder consumes current proprioception q_t and the ground-truth future action chunk A_t^E and infers a Gaussian posterior:

$$q_\phi(z \mid q_t, A_t^E) = \mathcal{N}\big(z; \mu_\phi(q_t, A_t^E), \operatorname{diag} \sigma_\phi^2(q_t, A_t^E)\big)\,.$$

The reparameterization trick $z = \mu_\phi + \sigma_\phi \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ enables gradient-based optimization. The decoder (the deployed policy) uses complete available observation and z:

$$\hat{A}_t = f_\theta(h_t, z), \qquad p(z) = \mathcal{N}(0, I).$$

For training efficiency, **the encoder in the original ACT does not consume the camera images**: it infers z primarily from the expert action sequence and proprioception, while visual information enters the decoder. The style variable should not be identified with a physical contact force, a task phase, or a task ID; it is also not an independent noise variable resampled for each action timestep.

The loss combines reconstruction and KL regularization:

$$\mathcal{L}_{\text{ACT}} = \frac{1}{|\mathcal{B}|} \sum_{(h, A^E) \in \mathcal{B}} \left[\frac{\sum_{k=0}^{H-1} m_k \, \|\hat{a}_{t+k|t} - a_{t+k}^E\|_1}{\sum_k m_k + \varepsilon} + \beta \, D_{\text{KL}}\big(q_\phi(z \mid q_t, A_t^E) \| \mathcal{N}(0, I)\big) \right].$$

The mask m_k excludes padding at episode ends. For a diagonal-Gaussian posterior,

$$D_{\text{KL}} = \frac{1}{2} \sum_{j=1}^{D_z} \big(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\big).$$

The reconstruction term rewards accurate imitation; β controls information retained in the latent code. Too little KL regularization may create a mismatch between posterior-conditioned training and prior-conditioned inference; too much may collapse the latent variable. Implementations can use L_1, L_2, or other action losses; the **original ACT paper describes an L_1 reconstruction loss in its implementation discussion**, so squared error is not ACT's uniquely defining loss.

4.2.3 Transformer information flow

The original design can be understood as two paths:

```
Training only:
  [joint state, ground-truth future actions]
       -> CVAE encoder -> (mu, sigma) -> z

Policy / inference:
  [multi-camera images -> visual feature tokens]
  [joint state] + [z] -> Transformer encoder (fusion)
                   -> Transformer decoder (H future queries)
                   -> H x D_a joint-position targets
```

A visual backbone maps each view to spatial feature tokens with positional embeddings. Joint state and latent style are embedded as additional tokens; the Transformer encoder fuses them. A decoder with H future-time query tokens cross-attends to this representation and predicts all H actions **in a single decoder pass**. The original design is not required to autoregressively sample one joint command at a time.

For action dimension D_a, output shape is $H \times D_a$. A larger H provides longer temporal coverage but also asks the network to predict further into uncertain future contacts and observations. New hardware requires compatible input/output dimensions and action semantics, not blind reuse of the original ALOHA arm dimensions.

4.2.4 Why training and deployment are asymmetric

During training, the expert future chunk is known and can be used by the CVAE encoder to infer $q_\phi(z \mid q_t, A_t^E)$ before applying the reconstruction and KL losses. **During deployment**, the future expert chunk is unavailable; the encoder is discarded. The original ACT chooses the prior mean $z = 0$, yielding a deterministic prediction $f_\theta(h_t, 0)$. Sampling $z \sim \mathcal{N}(0, I)$ is an optional generative variation, **not the default procedure in the original ACT implementation**.

Thus a latent-variable training objective does not imply stochastic real-robot behavior, and access to future actions during training does not imply access to future expert actions at inference.

4.2.5 Temporal Ensembling: averaging predictions for the same physical instant

Suppose repeated policy calls made at query times $r \in \mathcal{R}(t)$ all produced action chunks containing a prediction for physical time t, denoted $\hat{a}_{t|r}$. Temporal Ensembling combines predictions **for the same physical execution time**:

$$\bar{a}_t = \frac{\sum_{r \in \mathcal{R}(t)} w_{r,t}\, \hat{a}_{t|r}}{\sum_{r \in \mathcal{R}(t)} w_{r,t}}, \qquad w_{r,t} = \exp[-m\, j(r,t)].$$

Here $j(r,t) = 0, 1, \ldots$ orders predictions from oldest to newest, matching the original ACT weighting convention. With this indexing, larger m gives comparatively more weight to older plans (slower response), and smaller m incorporates new observations more quickly. **This is not an ordinary filter over neighboring execution times** a_{t-1}, a_t, a_{t+1}; it fuses multiple forecasts for one target time.

Ensembling is easiest when the model can be queried frequently enough. When policy inference takes longer than a control cycle, ensembling alone cannot ensure uninterrupted execution: this is

precisely the latency problem addressed by RTC in Chapter 5. Excessively conservative averaging can also wash out brief but essential contact transitions.

4.2.6 Applicability and failure modes

ACT is well suited to consistent, high-quality demonstrations, especially coordinated and phase-structured manipulation. Its main practical attraction is a **supervised, one-pass prediction of a coherent action chunk without iterative diffusion sampling**. It does not directly model contact forces, solve state estimation, retarget actions between robots, or guarantee recovery from distribution shift.

Failure modes include uncertain distant actions for large H, learner-induced states not covered by BC data, latent posterior collapse, smoothing across contact events, and mismatches between teleoperator labels and deployed command semantics. **ACT improves the action representation; it does not theoretically eliminate BC covariate shift.**

Reference: Zhao et al., *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT / ALOHA, 2023).

4.3 Diffusion Policy: iterative denoising

Noising a demonstration chunk A_0 at diffusion index n gives

$$A_n = \sqrt{\bar{\alpha}_n} A_0 + \sqrt{1 - \bar{\alpha}_n}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

A noise-prediction loss takes the form

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{A_0, h, n, \epsilon} \left[\| \epsilon - \epsilon_\theta(A_n, n, h) \|^2 \right].$$

Generation denoises from noise over multiple model evaluations. This models multimodal action sequences but requires attention to inference cost, chunk-boundary behavior, and latency.

4.4 Flow Matching: a velocity field in generation space

Choose a noise chunk $A_0 \sim p_0$ and data chunk $A_1 \sim p_{\text{data}}(\cdot \mid h)$. For a straight interpolation path,

$$A_s = (1 - s) A_0 + s A_1, \quad v^*(A_s, s, h) = A_1 - A_0,$$

and a conditional flow matching objective is

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{s, A_0, A_1, h} \| v_\theta(A_s, s, h) - (A_1 - A_0) \|^2.$$

To generate actions, numerically integrate $dA_s/ds = v_\theta(A_s, s, h)$. Here $s \in [0, 1]$ denotes **artificial generative time**, and v_θ is a **velocity in action-sample space**, not the physical joint velocity or end-effector twist. Implementations may employ other coupling schemes, interpolation paths, and solvers.

4.5 Choosing a policy family

Method	Output	Strength	Key cost
Single-step BC	One action	Simple, easy to evaluate at high frequency	Limited temporal coordination/multimodality
Chunked BC / ACT	Fixed-length action chunk	Temporal structure with amortized inference	Boundary transitions and open-loop commitment
Diffusion Policy	Generative chunk	Expressive multimodal distribution	Repeated denoising steps
Flow Matching	ODE-generated chunk	Continuous generative paths	Numerical integration and latency

References: ACT / ALOHA, Diffusion Policy, and Flow Matching.

5. Real-Time Action Execution and RTC

5.1 Two clocks: physical control and model inference

An action-chunking policy predicts commands **assigned to future physical execution times**, but inference itself consumes wall-clock time. Let the low-level control period be Δt_c and the end-to-end delay from an available observation to a usable new action chunk be T_{lat}. The corresponding delay measured in robot cycles is approximately

$$d = \left\lceil \frac{T_{\text{lat}}}{\Delta t_c} \right\rceil .$$

In an actual robot stack, T_{lat} must account for relevant observation availability, preprocessing, networking, model inference, return transport, and scheduling. GPU-only forward-pass time is not a substitute for physical reaction time. When delay is variable, d also varies; robust execution needs a budget for tail latency rather than relying indefinitely on a mean.

Let H denote the **prediction horizon**, and let $s \le H$ be the intended **execution horizon** before replacing a chunk. Even if $H > s$, the executor cannot simply obtain missing actions from the next chunk while its inference has not finished.

5.2 Synchronous execution, naive asynchrony, Temporal Ensembling, and RTC

Mechanism	What happens during inference?	How are boundaries handled?
Synchronous execution	Wait or hold the old setpoint	Pauses; continuity not assured in a dynamic environment
Naive asynchrony	Keep sending actions from the old chunk	A discontinuity can occur when the new chunk arrives
Temporal Ensembling	Aggregate multiple predictions for one execution instant	Smooths by averaging; does not constrain generation to the committed prefix
RTC	Execute the old chunk while generating a compatible new chunk	Uses overlap actions to condition or guide new generation

Temporal Ensembling is **an output aggregation rule commonly associated with ACT**. RTC is **a generation-and-execution protocol for nonzero inference latency**. They are not equivalent. Any execution method should schedule actions against physical timestamps and reject or safely handle stale commands.

5.3 The RTC timeline: committed prefix and unconstrained suffix

Suppose the previous chunk started at step $t - s$ and covers $[t - s, t - s + H)$. A new inference call starts at step t and is expected to finish at $t + d$. While that call runs, the robot executes actions from the old chunk. The new chunk corresponds to $[t, t + H)$, so its first d steps overlap already committed execution.

If the old chunk has local indices $0, \dots, H - 1$, its actions aligned with the new chunk's committed prefix are

$$b_i = A^{\text{old}}_{s+i}, \qquad i = 0, \dots, d - 1.$$

A basic condition for this overlap to exist is

$$0 \le d \le H - s.$$

This condition matters: if $d > H - s$, the old chunk cannot completely cover the waiting interval. The system must request less-frequent updates, lengthen the prediction horizon, reduce latency, or hand control to a bounded low-level fallback. **RTC does not invent physically safe commands when no valid queued actions remain.**

For an indexing illustration only (not an experiment), choose $H = 16$, $s = 6$, and $d = 3$. Old actions $6, 7, 8$ cover the first three slots of the new chunk; only when the new output becomes available at $t + 3$ may its slot 3 begin execution.

```
physical step     t       t+1     t+2     t+3      t+4  --->
robot command     old[6]  old[7]  old[8]  new[3]   new[4]
new chunk index     0       1       2       3        4
status           committed prefix  |    free suffix
model              <--- inference --->
```

The conceptual generation objective is

$$A^{\text{new}}_{0:d-1} \approx b_{0:d-1}, \qquad A^{\text{new}}_{d:H-1} \text{ consistent with observation and prefix.}$$

The symbol $\approx$ denotes approximate consistency **of the generated chunk**. Already executed old commands cannot be overwritten. Original inference-time RTC uses guided inpainting to approximate this objective; it is not an exact computation of an arbitrary model's conditional distribution.

5.4 Inference-time RTC: generate a new chunk through inpainting

Black, Galliker, and Levine (2025) show how to guide an existing diffusion- or flow-based action policy **without retraining**. A simple hard mask emphasizes the first d committed actions. However, matching only those steps may still permit a sharp change in direction after the prefix. Their **soft masking** additionally uses the overlapping but uncommitted future actions as gradually weaker preferences.

Let Y_i be the old chunk's action aligned to index i on the new timeline, and let $M_i \in [0,1]$ encode consistency strength. A useful **pedagogical abstraction** is

$$\mathcal{L}_{\text{match}}(A) = \frac{1}{2}\sum_{i=0}^{H-1} M_i \left\|A_i - Y_i\right\|^2, \qquad M_i = \begin{cases} 1, & 0 \le i < d, \\ \rho_i, & d \le i < H-s, \\ 0, & H-s \le i < H, \end{cases}$$

where ρ_i decreases from near one toward zero. Committed actions are strongly guided, distant overlapping predictions weakly guided, and entirely new future slots remain free to react to updated observations. **RTC is not merely interpolation between two fully predicted chunks: compatibility is imposed while generating the new chunk.**

For a flow model, let x_τ denote the intermediate chunk at generative time $\tau \in [0,1]$, and let its velocity field be $v_\theta(x_\tau, \tau, h_t)$. A straight-flow estimate of the final generated chunk is

$$\hat{A}_1(x_\tau) = x_\tau + (1-\tau) v_\theta(x_\tau, \tau, h_t).$$

A conceptual consistency-guided velocity field is

$$\tilde{v}(x_\tau) = v_\theta(x_\tau, \tau, h_t) - \lambda_\tau \nabla_{x_\tau} \mathcal{L}_{\text{match}}\big(\hat{A}_1(x_\tau)\big).$$

The actual RTC paper uses pseudoinverse-based guidance (ΠGDM), soft weights, and guidance clipping. The equation above is a **mechanism-level illustration, not a verbatim reconstruction of every coefficient in the original algorithm**. Evaluating the guidance requires a vector-Jacobian product (VJP) through the final-action estimate, creating **additional reverse-mode work for each denoising/integration step**. Aggressive guidance with few denoising steps may be unstable, making clipping important.

Inference-time RTC can be added to compatible pretrained diffusion/flow policies. Its limitations include guidance overhead, delay estimation, and inpainting approximation. An ordinary deterministic ACT decoder does **not** support the same flow-guidance formula without redesign.

5.5 Training-time RTC: learn to condition directly on the action prefix

Black et al. (2025) instead **simulate inference delay during training** so that prefix conditioning is learned directly, avoiding inference-time gradient-based inpainting. For a demonstrated action chunk $A^E = [a_0^E, \dots, a_{H-1}^E]$, randomly sample a prefix length d. The first d actions remain known and uncorrupted; only the remaining $H-d$ actions are noised and supervised using Flow Matching.

Let $\tau \in [0,1]$ denote generative time and $\epsilon_i \sim \mathcal{N}(0, I)$. Form the token-wise model inputs

$$x_i^\tau = \begin{cases} a_i^E, & i < d, \\ (1-\tau)\epsilon_i + \tau a_i^E, & i \ge d, \end{cases} \qquad r_i = \begin{cases} 1, & i < d, \\ \tau, & i \ge d. \end{cases}$$

Here r_i is a **per-action-token flow-time condition** indicating that the prefix token is already fully resolved. Mask the objective to supervise the suffix only:

$$\mathcal{L}_{\text{TT-RTC}} = \mathbb{E}\left[\frac{1}{H-d}\sum_{i=d}^{H-1} \left\| v_\theta(x^\tau, r, h)_i - (a_i^E - \epsilon_i) \right\|^2\right], \qquad 0 \le d < H.$$

At deployment, insert the previous chunk's committed actions as the known prefix, keep their times at $r_i = 1$, and generate only the unknown suffix. No VJP-based inpainting correction is needed at each

integration step. Models originally using one shared flow timestep may need a **per-token timestep embedding and suffix loss mask**. The paper notes that this can generally be achieved without new learnable parameters, not that every existing implementation has zero interface changes.

Attribute	Inference-time RTC	Training-time RTC
Retrain the pretrained policy?	Typically no	Requires conditioning training or fine-tuning
Primary guidance	Hard prefix + soft overlap	Known hard prefix + supervised suffix
Extra inference work	VJP-based guidance	No additional inpainting backpropagation
Latency adaptation	Adjust mask/delay estimate at inference	Depends on delay distribution covered in training
Model interface	Diffusion/flow with generation-time guidance	Per-token time conditioning and prefix masking

Training sees **ground-truth expert prefixes**, whereas deployment prefixes are generally predicted by an earlier model call; this may create a further **prefix-distribution shift**. This is an important practical caveat, not an outcome proved away by the training-time RTC paper. Training-time conditioning is also less flexible than inference-time soft guidance over the entire overlapping future.

5.6 Runtime components needed by an RTC executor

A safe execution interface should separate the following responsibilities (these are architectural requirements, not an experimental procedure):

1. **Control clock and command queue.** A low-level controller consumes committed actions at a fixed rate, independent of whether high-level inference is running.
2. **Timestamped observations and plans.** Requests carry observation availability time, planned execution origin, and expected deadline; late responses are checked before use.
3. **Delay estimate and aligned prefix.** Estimate d from recent end-to-end delays, align future actions from the active queue, and reserve clock/jitter margin.
4. **Atomic plan switching.** Update the source of commands at one defined physical time, avoiding race conditions where multiple inference workers overwrite the queue.
5. **Safety fallback.** On deadline misses, queue exhaustion, invalid observations, or dangerous contacts, trigger bounded holding, slowdown, stop, or a separate recovery controller, according to the robot's dynamics and command semantics.

Action timestamps are more important than inference request IDs. If a supposedly d-cycle response actually returns at $t + d + 2$, the executor must not apply chunk index d as the current action; it must discard expired slots, realign safely, or activate fallback.

5.7 What RTC does not solve: contact, perception, and coverage

RTC addresses **continued execution under computation delay and compatibility across chunk boundaries**. It does not ensure physical safety or semantic correctness of generated actions. Even if one enforces

$$\|u_{t+1} - u_t\| \le \delta_u,$$

high-stiffness contacts can still produce unsafe forces. Conversely, certain grasp, release, or mode-switch events genuinely require sharp transitions and should not be uniformly low-pass filtered. Position, velocity, torque, and tactile control have different allowable transitions.

RTC does not eliminate BC covariate shift, fix camera occlusion, estimate unknown friction from nothing, or create reliable dropped-object retrieval skills. Those issues require better sensing and state estimation, better data coverage, interactive corrections, feedback control, or high-level replanning.

5.8 Concept map

Action Chunking specifies how many future actions are predicted at once. **ACT** is one Transformer/CVAE architecture for those chunks. **Temporal Ensembling** aggregates predictions of the same physical action from overlapping chunks. **Asynchronous execution** keeps the low-level controller running while inference is in progress. **RTC** further makes new chunks compatible with the commands already committed. **Training-time RTC** learns this prefix conditioning during training. These concepts live at different levels—policy representation, aggregation, scheduling, and training—and should be combined deliberately rather than treated as interchangeable algorithms.

References: Real-Time Execution of Action Chunking Flow Policies (NeurIPS 2025); Training-Time Action Conditioning for Efficient Real-Time Chunking (2025).

6. Augmentation, Retargeting, and Recovery

6.1 Physically consistent augmentation

Augmentation should transform object configurations, reference trajectories, and labels consistently rather than arbitrarily modifying visual frames. A rigid scene transform $G \in SE(3)$ might transform an object pose and end-effector reference together, but robot joint limits, base geometry, gravity, and collisions may invalidate the result.

One can express a valid augmentation as

$$\tau' = \mathcal{T}(\tau, \xi), \quad \text{s.t. } c_{\text{kin}}(\tau') \le 0,\ c_{\text{col}}(\tau') \le 0,\ c_{\text{contact}}(\tau') \le 0.$$

Here the constraints cover kinematics, collision, and contact consistency. Geometric feasibility alone does not imply dynamic feasibility, especially around contact changes. See MimicGen and DexMimicGen.

6.2 The missing information in human-video retargeting

Human-video-to-robot pipelines combine 3D hand/object reconstruction, trajectory smoothing, scale/frame alignment, candidate contact segmentation, robot IK or joint retargeting, and physical feasibility correction. Vision alone generally does **not** uniquely determine hidden contact forces, friction, or intent under occlusion.

For dexterous manipulation, useful event classes include first contact, stable grasp, slip, regrasp, release, drop, and recovery. Combine image cues with joint feedback, motor current/torque, and tactile sensing where available; preserve uncertainty in weak event annotations.

6.3 Learning from failure without copying failure

Distinguish (i) valid actions before failure, (ii) erroneous actions that should **not** be copied, and (iii) expert recovery/correction actions. Naïvely treating entire failed episodes as positive BC demonstrations reinforces bad decisions. Human corrections can train recovery behavior; expert relabeling on learner states supports DAgger; reward/value-based improvement belongs to reinforcement learning.

The data loop is

$$\mathcal{D}_0 \xrightarrow{\text{train}} \pi_0 \xrightarrow{\text{deploy/observe}} \mathcal{F}_0 \xrightarrow{\text{correct/relabel}} \Delta\mathcal{D}_0 \xrightarrow{\text{curate}} \mathcal{D}_1.$$

Log provenance, demonstrator identity, model version, and intervention status so that unverified model-generated commands never silently become ground-truth expert actions.

7. Deployment Interfaces and Course Boundaries

7.1 Modular contracts

A maintainable system separates **perception/state estimation** (timestamped sensory representations), **policy inference** (task-conditioned action chunks), **real-time executor** (buffering, scheduling, RTC), **safety/control layer** (saturation, collision avoidance, emergency stop), and **logging** (predictions, commands, execution feedback). A clearly defined interface is often more important than which model family supplies the action chunk.

7.2 Diagnosing common failure modes

Symptom	First checks
Good offline loss, poor closed-loop behavior	Covariate shift, sensor latency, frame mismatch
Pauses or jerks at chunk boundaries	Blocking inference, unaligned physical times, prefix mismatch
Contact action is averaged away	Temporal ensembling, incompatible modes in labels
Weak object generalization	Split leakage, insufficient object/initial-state coverage
Recovery fails repeatedly	Invalid failure labels, missing expert corrections, broken estimation
Motion in wrong direction or scale	Units, frames, relative vs. absolute commands, normalizer version

7.3 Relationship to other course modules

These notes address **how demonstrations become reliable supervised actions and how chunked actions execute on a real timeline**. Camera calibration and visual state estimation cover cross-frame geometry and occlusion handling; the Flow Matching module develops transport paths and solvers; the RL course covers reward-based policy optimization; the Robot Agent module covers long-horizon planning, failure detection, and subtask replanning. Clearly defined contracts prevent duplicated theory across modules.

References and Further Reading

1. Ross et al., *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning* (DAgger, 2011). https://proceedings.mlr.press/v15/ross11a.html
2. Zhao et al., *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT / ALOHA, 2023). https://arxiv.org/abs/2304.13705
3. Chi et al., *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion* (2023). https://arxiv.org/abs/2303.04137
4. Lipman et al., *Flow Matching for Generative Modeling* (2022/2023). https://arxiv.org/abs/2210.02747
5. Wu et al., *GELLO: A General, Low-Cost, and Intuitive Teleoperation Framework for Robot Manipulators* (2023). https://arxiv.org/abs/2309.13037
6. Chi et al., *Universal Manipulation Interface* (2024). https://arxiv.org/abs/2402.10329
7. Mandlekar et al., *MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations* (2023). https://proceedings.mlr.press/v229/mandlekar23a.html
8. *DexMimicGen* project. https://dexmimicgen.github.io/
9. Black, Galliker & Levine, *Real-Time Execution of Action Chunking Flow Policies* (2025). https://arxiv.org/abs/2506.07339
10. Black et al., *Training-Time Action Conditioning for Efficient Real-Time Chunking* (2025). https://arxiv.org/abs/2512.05964
11. LeRobot Dataset v3 documentation. https://huggingface.co/docs/lerobot/lerobot-dataset-v3

Editorial note: This is an instructional rewrite of the existing course topic. Equations are standard or pedagogical abstractions, not exact implementations of every cited method. No experiments or robot validations are claimed or included.